Project Synopsis

AMITY UNIVERSITY, MUMBAI

Amity School of Engineering and Technology

Department of Computer Science & Engineering

MINI PROJECT SYNOPSIS

(Academic Year 2025–26)

── SMART CALORIE & NUTRITION TRACKER ──

Submitted By:

1. Niraj Vaidya (Enrl No: A70405223113)
2. Diwkar Rai (Enrl No: A70405223026)
3. P. Jaswant Rao (Enrl No: A70405223030)

Under the Guidance of:

Name of Guide: Dr. Dipak Raskar
Designation: Professor

1. Title of the Project

Smart Calorie & Nutrition Tracker

2. Abstract

In today's fast-paced world, maintaining a healthy lifestyle and tracking dietary intake can be challenging. To address this, the Smart Calorie & Nutrition Tracker proposes a modern, mobile-first Progressive Web Application designed to simplify daily food and macronutrient monitoring. The application allows users to log meals through three complementary low-friction workflows: an **AI vision-based food scanner** powered by on-device Transformers.js models, a **real-time barcode scanner** using `html5-qrcode`, and a **text search** powered by the **Open Food Facts** public database. Beyond meal logging, the app tracks **water intake**, **weight history**, and maintains **gamified daily streaks**, while automatically computing personalised calorie targets from the user's body profile using the **Mifflin–St Jeor BMR/TDEE equation**. Progress is visualised through an animated calorie ring, macro bars, and a Recharts-driven dashboard. Built with Next.js 14 and Firebase (Auth + Firestore), the app ensures real-time cross-device synchronisation, secure Google Sign-In, and a responsive glassmorphism-inspired dark-mode UI. The expected outcome is a fast, intuitive, and reliable tool that empowers users to make informed, healthier dietary choices with minimal effort.

3. Problem Statement

Maintaining an accurate record of daily food consumption is often tedious and time-consuming, leading to low long-term adherence among individuals trying to eat healthier. Standard applications often have cluttered interfaces, require manual search for every food item, or lock essential features like barcode scanning behind expensive paywalls. There is a need for a streamlined, accessible, and fast tool that minimizes the effort required to log food and track macronutrients (protein, carbs, and fats).

4. Objectives

- **Objective 1:** To develop a user-friendly, responsive Progressive Web Application (PWA) with a premium dark-mode glassmorphism aesthetic for tracking daily caloric and macronutrient intake.
- **Objective 2:** To implement three complementary low-friction food-logging paths — an **AI camera scanner** (client-side Transformers.js vision model), a **barcode scanner** (`html5-qrcode`), and a **text search** against the Open Food Facts database — to drastically reduce the time needed to log meals.
- **Objective 3:** To automate goal-setting by deriving personalised daily calorie targets from the user's body profile via the **Mifflin–St Jeor BMR equation**, an activity multiplier (TDEE), and a weight-goal offset (lose / maintain / gain).
- **Objective 4:** To provide real-time dashboard analytics with Recharts-powered visualisations (calorie ring, macro bars, water, weight, and historical trends) for immediate feedback on nutritional progress.
- **Objective 5:** To gamify adherence through a **consecutive-day streak** system that rewards consistent logging.

- **Objective 6:** To integrate cloud database synchronisation via Firebase Authentication (Google Sign-In) and Cloud Firestore for persistent, secure, multi-device storage without data loss.

5. Scope of the Project

The project includes Google Sign-In authentication, automated personalised daily calorie goals (BMR/TDEE-driven), real-time nutrient tracking (calories, protein, fats, carbohydrates), water tracking, weight logging, a saved "My Foods" library, an integrated barcode scanner, an AI camera scan, a Recharts-based analytics dashboard, a history view, and gamified streaks. **Limitations:** The application relies on an active internet connection to query the Open Food Facts API and sync with Firebase. Camera, AI inference, and barcode functionality depend on device hardware (camera availability, sufficient WebGPU/WASM performance for client-side models) and browser permissions. Medical or highly specialised dietary planning (e.g., diabetic tracking, micronutrient analysis) is beyond the current scope.

6. Literature Survey / Existing System

Existing health tracking systems like MyFitnessPal, LoseIt!, and Cronometer offer extensive tracking features but suffer from several drawbacks. Many of these platforms have increasingly monetised basic functionalities, effectively locking free barcode scanning and macronutrient details behind premium subscriptions. Their interfaces can also be overwhelming for casual users who just need a quick, distraction-free logging experience. This project improves upon existing systems by offering a lightweight, core-focused feature set with **free** barcode scanning, **on-device AI food recognition**, automated goal calculation, and a clean, ad-free, premium dark-mode UI.

7. Proposed System

The proposed system is a Next.js 14 (App Router) web application that specifically targets the friction of dietary tracking. It features an intuitive dashboard with an animated calorie ring, macro bars, water tracker, and collapsible meal sections. Users can add food through three complementary paths: an **AI camera scan** (client-side Transformers.js model), a **barcode scan** (`html5-qrcode` → Open Food Facts), or a **text search** (Open Food Facts). The Settings page captures the user's body profile and auto-derives the daily calorie goal via the Mifflin–St Jeor equation. A separate Dashboard route uses Recharts to visualise calorie, macro, water, and weight history. By relying on Firebase Authentication and Firestore, the app delivers instant cross-device sync and secure auth without a custom REST backend, producing a lightweight native-app-like experience in the browser.

8. Methodology

- **Requirement Analysis:** Identified user pain points with existing apps; gathered functional requirements such as authentication, multi-modal food logging, automated goal computation, water/weight tracking, gamified streaks, and visual analytics.

- **Design (UML Diagrams):** Architected the component hierarchy under the Next.js App Router (`/`, `/dashboard`, `/history`, `/my-foods`, `/settings`) and designed the Firestore document model (`users/{uid}`, `users/{uid}/logs/{date}`, `users/{uid}/logs/{date}/entries/{id}`, `users/{uid}/myFoods/{id}`). Produced UI/UX wireframes emphasising a glassmorphism dark-mode aesthetic with a fixed bottom navigation bar.
- **Development:** Built the frontend with Next.js 14 and React (CSS Modules + global CSS variables). Integrated Firebase Authentication (Google Sign-In) via a global `AuthContext`, and Firestore for real-time CRUD on food entries, logs, settings, water, weight, and saved foods. Integrated `html5-qrcode` for barcode scanning, **Transformers.js** for on-device food image classification, the **Open Food Facts API** for nutrition data, and **Recharts** for the analytics dashboard. Implemented the Mifflin–St Jeor BMR/TDEE goal engine and the consecutive-day streak logic in the service layer.
- **Testing:** Conducted component-level layout testing, verified Firestore Security Rules for per-user read/write isolation, validated serving-size scaling math, and performed cross-browser and responsive testing across mobile and desktop.

9. Tools & Technologies

Programming Language: JavaScript (ES6+), HTML5, CSS3 **Framework:** Next.js 14 (App Router), React **Database & Auth:** Firebase Cloud Firestore, Firebase Authentication (Google Sign-In) **AI / Scanning:** Transformers.js (Hugging Face, client-side), `html5-qrcode` **External API:** Open Food Facts REST API **Visualisation:** Recharts **Hosting & Tooling:** Firebase App Hosting / Firebase Hosting, Git/GitHub

10. System Requirements

Hardware: Any smartphone, tablet, or PC with a modern web browser and a functioning camera (for barcode scanning).
Software: Any modern web browser (Google Chrome, Apple Safari, Mozilla Firefox, Microsoft Edge).

11. Expected Outcomes

The final deliverable will be a fully functional, publicly hosted Progressive Web Application. Users will be able to sign in with Google, set up a body profile that auto-derives a personalised daily calorie goal, and effortlessly track their daily intake by **photographing meals (AI scan), scanning barcodes, or searching the Open Food Facts database**. They will additionally be able to log water and weight, view rich Recharts visualisations on a dashboard, browse historical days, save reusable custom foods, and stay engaged through a consecutive-day streak system. The overarching benefit is increased awareness of dietary habits through a seamless, free, ad-free UI experience — without the subscription paywalls that gate competing apps.

12. Timeline / Work Plan (January 3rd week to May 1st week)

Phase	Activity	Duration
1	Requirement Analysis & UI Prototyping	2 Weeks
2	Architecture Design & Database Setup	2 Weeks
3	Core Development & Feature Integration (Next.js, Firebase, Barcode Scanner)	6 Weeks
4	Testing, Debugging, Deployment, & Documentation	4 Weeks

13. References

[1] Next.js Documentation. [Online]. Available: https://nextjs.org/docs [2] Firebase Documentation. [Online]. Available: https://firebase.google.com/docs [3] React.js Official Documentation. [Online]. Available: https://react.dev/ [4] Mebjas, “html5-qrcode,” GitHub. [Online]. Available: https://github.com/mebjas/html5-qrcode [5] Open Food Facts API Documentation. [Online]. Available: https://world.openfoodfacts.org/data [6] Hugging Face, “Transformers.js,” [Online]. Available: https://huggingface.co/docs/transformers.js [7] Recharts Documentation. [Online]. Available: https://recharts.org/ [8] Mifflin MD, St Jeor ST, et al., “A new predictive equation for resting energy expenditure in healthy individuals,” *American Journal of Clinical Nutrition*, 1990.